Error Reporting System (ERS) — Service Study

DAQ-WP1 — service analysis for the AI DAQ-agent project

Sources (all directly inspected, not just described):

1. `atlas-tdaq-software/ers` — real ATLAS source, `python/ers.py`
2. `atlas-tdaq-software/mts` — README, release notes, OKS schema, and the actual `mts2splunk_receiver.cpp` + `ers2splunk.sh` source (the Splunk ingestion pipeline itself)
3. `atlas-tdaq-software/pbeast` — README + `python/pbeastpy.cpp` (archive for a *related* DAQ service — see note in §3)
4. OH User API: How to use the ERS package (full content, real ATLAS docs)
5. Kolos, Kazarov, Papaevgeniou — *The Error Reporting in the ATLAS TDAQ System* (ATL-DAQ-PROC-2014-027)
6. *Experience with SPLUNK for Archiving and Visualisation of Operational Data in ATLAS TDAQ System* (ATL-DAQ-SLIDE-2018-087)
7. TDAQ official release package list
8. Live infrastructure host-status page (confirms Splunk infra for TDAQ running as of 2025)

Note: `gitlab.cern.ch/atlas-tdaq-software` is publicly browsable — no CERN account needed.

---

1. What ERS is

ERS is how every application in ATLAS's DAQ system reports problems — errors, warnings, debug info. Every application uses the same API and data model. An app creates a typed **Issue** and sends it to a severity stream (`DEBUG`/`LOG`/`INFO`/`WARNING`/`ERROR`/`FATAL`); where it ends up depends entirely on configuration.

---

2. The data model

- **Issue** — every kind of error is its own typed subclass, not a plain string. Defaults to `ERROR` severity until sent to a stream.

- **Context** — auto-recorded: package, file, function, line, host, user, process ID, application name.
- **Qualifiers** — filter tags, defaulting to the raising package's name.
- **Severity** — `DEBUG`, `LOG`, `INFO`, `WARNING`, `ERROR`, `FATAL`.

Routing config, per severity, via environment variable:

export TDAQ_ERS_ERROR="lstderr,filter(foo,bar),throw"

Building blocks: `stdout`/`stderr`/`lstdout`/`lstderr`, `null`, `throw`, `abort`, `exit`, `filter(...)`/`rfilter(...)`, `throttle(threshold, interval)`.

---

3. How to get the data — the full, confirmed pipeline

A. Live delivery — MTS, confirmed C++/Java only

Live delivery runs through **MTS**, a CORBA-based pub/sub transport, plugged in as an ERS stream (`TDAQ_ERS_ERROR='lstderr,mts'`), with a real filter language (by app, message, severity, custom parameter, or context field).

Ready-made tools: `mts_ers_receiver -p $TDAQ_PARTITION -s "sev=ERROR AND ..." -v` (CLI) and `start_ers_monitor` (GUI).

Python confirmation, fully settled: Java has a documented receiver class (`MTSInStream extends ers.AbstractInputStream`); the MTS README states "C++ and Java" explicitly; the TDAQ package list separates out "Jers – Java ERS" with no Python equivalent; and `ers.py` only implements sending. No Python live-subscribe exists.

B. Historical archive — Splunk, fully confirmed end-to-end including exact message format

The real ingestion tool, `mts2splunk_receiver`, was directly inspected. Here's exactly how it works:

Pipeline:

App raises Issue → ERS stream → MTS → mts2splunk_receiver subscribes to ALL messages
  → buffers to a temp file (/tmp/ers.<PID>.out)
  → every N seconds (default 5s), closes & moves it to /data/4splunk/<partition>.<timestamp>.out

→ Splunk asynchronously monitors, indexes, and removes files from /data/4splunk/

Alternatively (`ers2splunk.sh`), the same receiver's output can be piped live over `nc host port` straight into a Splunk TCP listener instead of writing files — this is the testbed variant from the release notes, now confirmed with exact command:

mts2splunk_receiver -d -s "$subscription" -p "$partition" -i "stdout" | nc "$host" "$port"

The exact archived record format (confirmed straight from the source comment — this is real, not inferred):

t=1463153399, rn=299119, part=IBL-DAQSlice, uname=crpixel, msgID=rc::ApplicationSignaled, host=pc-pix-fit-02.cern.ch, app=Ctrl_PixelIBL, sev=WARNING,
text="Application ... died while exiting on signal 9.",
context="PACKAGE_NAME: RunControl. FILE_NAME: .../ApplicationController.cxx. FUNCTION_NAME: ...
LINE_NUMBER: 182. DATE_TIME: 1463153399.",
params="application: ... errStream: ... hostName: ... onExit: 1 ...",
quals="RunControl PIXEL ", chained="0", gh=123244, T0=data_test, LB=45, Fill=0, R4P=0, SB=0

Field	Meaning
`t`	Unix timestamp
`rn`	**Run number** — this answers the earlier open question about run linkage directly
`part`	Partition name
`uname`	User who ran the app
`msgID`	Issue class name (the "type" of the error, e.g. `rc::ApplicationSignaled`)
`host`	Host where the issue occurred
`app`	Application name
`sev`	Severity
`text`	The message

`context`	Package, file, function, line, timestamp of the raise-site
`params`	Issue's custom key/value attributes
`quals`	Qualifiers
`chained`	0 = standalone, 1 = top of a chain, 2 = nested cause
`gh`	Group hash — links a chained set of issues together
`T0`	Tier-0 processing project tag
`LB`	Luminosity block number
`Fill`	LHC fill number
`R4P`	"Ready for physics" flag
`SB`	"Stable beams" flag

Interesting detail: **run number, fill number, Tier-0 tag, luminosity block, and the R4P/SB flags are not part of the ERS Issue itself** — the receiver actively looks them up separately from the **Information Service (IS)** and stamps them onto each archived record. This means the archive is already a small join between ERS and IS data, done at ingestion time — worth knowing, since it means your agent doesn't need to do that join itself for these fields, they're already present per-message.

Splunk-side config is documented too: field extraction rules live in `/opt/splunk/etc/apps/ers/default/props.conf` and `transforms.conf` — confirming the Splunk app is literally named `ers`.

The endpoints — now confirmed directly, closing the last open item:

- **Production**: `https://atlasop.cern.ch/tdaq/ers/en-US/static/app/ers/ers.html` — reached through `atlasop.cern.ch` acting as a reverse proxy (path `/tdaq/ers/...`). The real backend host/port is deliberately hidden behind this proxy — standard practice, not a gap in the research. This independently confirms the Splunk app name `ers`, matching the `props.conf`/`transforms.conf` path found in source.
- **Test/staging**: `https://vm-ers-test.cern.ch:8000/en-US/app/ers/ers_browser_html` — a directly-reachable Splunk instance, port 8000 (Splunk's standard web UI port), no proxy in front.

- Both found via CERN's internal TWiki oncall-shortcuts page (`PointOneDataFlow`), which is only reachable from the CERN network or via VPN — consistent with everything else about this pipeline being internal-only by design.

For a Python client, the practical target is almost certainly Splunk's REST API on the equivalent management port (typically 8089 for API access, separate from the 8000 web UI port) against whichever of these two hosts you're authorized to query — worth confirming the exact API port and an access token/credential with your CERN contact, but the *host* question itself is now answered.

Worth noting: `mts2splunk_receiver` supports dropping specific `msgId`/`appId` values at ingestion (`-m`, `-a` flags) — so "the archive" may not be 100% complete by design in a given deployment.

A useful side-note: P-BEAST (a related, but different, service)

`atlas-tdaq-software/pbeast` archives the **Information Service (IS)**, not ERS. Useful as an architecture reference (Python `ServerProxy` class, CERN-SSO-authenticated), and directly relevant to whichever teammate is studying IS — note that IS is also the source of the `rn`/`Fill`/`T0`/`LB`/`R4P`/`SB` fields the ERS archive borrows.

---

4. The real Python API for writing Issues (`ers.py`)

- `Severity` — 6 levels as integers 0–5.
- `Context` — built automatically from the call stack.
- `Issue(Exception)` — base class: `.severity`, `.message`, `.cause`, `.context`, `.parameters`.
- Sending functions: `debug(lvl, msg)`, `log(msg)`, `info(msg)`, `warning(issue)`, `error(issue)`, `fatal(issue)`.
- Wraps a compiled library (`liberspy`) — a real, first-class binding.
- Bridges into Python's standard `logging` module (`LoggingHandler`, `addLoggingHandler()`).

This is the write side — the only side with a real Python implementation.

---

5. Scale

- Peak rate: **~10,000 messages/second** during data-taking.
- Total archived in Run 1: **~30 million messages**.
- Some messages may be excluded from the archive by design (`-m`/`-a` drop flags at ingestion).

Implication: filter by run, time window, severity, source app, and keyword before anything reaches the agent — never expose raw access.

---

6. What's genuinely still open

1. **The Splunk REST API port and an access credential/token.** The web UI hosts are now confirmed (`atlasop.cern.ch/tdaq/ers/...` for production, `vm-ers-test.cern.ch:8000` for test) — what's left is the API-specific port (likely 8089, standard for Splunk, but unconfirmed here) and how to authenticate a script against it (CERN SSO, a service account, or a token). This is a narrow, practical setup question, not an unknown.

Everything else previously open — archive backend, exact schema, ingestion mechanism, live-subscribe language support, run linkage, and now the endpoint itself — is resolved with real, directly-inspected sources.

---

7. Bottom line for WP2

- ERS data is clean, typed, and now fully mapped end-to-end — including the exact field names your agent's query results will actually contain (`t`, `rn`, `part`, `msgID`, `host`, `app`, `sev`, `text`, `context`, `params`, `quals`, `chained`, `gh`, plus IS-sourced enrichment fields `T0`/`LB`/`Fill`/`R4P`/`SB`).
- **Live delivery** = MTS, C++/Java only. `mts_ers_receiver` CLI is usable today as a stopgap; a native Python agent would need a small bridge.
- **Historical archive** = Splunk, app name `ers`, fed via `mts2splunk_receiver`/`ers2splunk` as a standard per-partition setup-segment process. Production web endpoint confirmed at `atlasop.cern.ch/tdaq/ers/...`; test instance directly reachable at `vm-ers-test.cern.ch:8000`.
- The archive already joins in IS data (run/fill/luminosity-block/beam-status) per message — your agent gets that "for free," no separate IS lookup needed for those fields.
- Python has a solid API for **writing**; for **reading**, plan a Splunk REST client against the confirmed host, once you have the API port and a credential from your CERN contact —

worth modeling on P-BEAST's `ServerProxy` pattern for consistency with the rest of your team's agents.
- Build filtering into the tool from day one.